

2025. 제67회 경남과학전람회

[작품설명서]

작품명 : 암호학의 역사를 바꾸는 테트레이션 연산 기반 암호화 연구

구분	학생부	출품 부문	산업 및 에너지
----	-----	-------	----------

2025. 07. 14

	소 속	성 명
출품자 (팀명: 슈퍼쑈니)	창원과학고등학교	권동연
	창원과학고등학교	김시현
	창원과학고등학교	안예서
지도교원	창원과학고등학교	손혜선

<작성 안내>

- 구분: 학생부, 교원부
- 출품 부문: 물리, 화학, 생물, 지구 및 환경, 산업 및 에너지(SW-IT 융합) 중 택 1
- **작품설명서는 A4 용지 30쪽 이내**로 작성하여 좌철 제본(30쪽 초과 시 초과 분량은 심사에서 제외)
- 본문에는 개인정보 또는 소속을 알 수 있는 내용(이름, 소속 학교, 학년, 개인을 식별할 수 있는 사진, 소속 학교를 식별할 수 있는 사진 등)이 포함되지 않아야 함
- 작품설명서 4부 중에서 표지에 인적사항 기재 1부, 인적사항 미기재 3부로 제출할 것

1. 연구 동기 및 목적

현대 사회에서 정보 보안은 점점 더 중요해지고 있으며, 이를 위해 다양한 암호화 기법이 개발되고 있다. 그러나 기존의 암호화 기법들은 일정한 수학적 원리에 기반하고 있어, 시간이 지나면서 이를 해독할 수 있는 알고리즘이나 기술이 발전할 가능성이 존재한다. 따라서 새로운 방식의 암호화 기법을 연구하는 것은 정보 보호의 지속성을 보장하는 중요한 요소가 된다. 본 연구에서는 기존의 암호화 방식과는 차별화된 접근법으로 테트레이션이라는 수학적 연산을 활용하여 새로운 암호화 방식을 연구하고자 한다.

테트레이션의 수렴과 발산 특성을 활용하여 새로운 방식의 암호화 키를 생성할 수 있으며, 이를 응용하여 암호화된 데이터를 보다 안전하게 보호할 수 있다. 테트레이션은 비선형적이고 복잡한 구조를 가지므로, 이를 기반으로 생성된 키는 일반적인 암호 해독 기법을 사용하여 분석하기 어렵다. 또한, 시각적 암호화 기법은 단순한 수학적 연산을 넘어 프랙탈 기반의 정보 은닉 기술을 적용할 수 있는 가능성을 제공한다.

2. 연구 내용: 연구할 내용

가. 이론적 배경

1) 암호

암호화는 크게 암호화 키와 복호화 키가 같은 방식인 비밀키(대칭키) 방식과 해쉬함수, 암호화 키와 복호화 키가 다른 방식인 공개키(비대칭키) 방식으로 나눌 수 있다.



그림 1 암호화 알고리즘의 분류

2) 테트레이션(tetration)

가) 테트레이션이란?

$${}^n a = a^{a^{\dots^a}}$$

테트레이션은 덧셈, 곱셈, 거듭제곱에 이어 등장하는 네 번째 하이퍼 연산으로, 거듭제곱을 반복하는 방식으로 정의된다. 자연수 n 의 경우, 이전 연산을 n 번 반복하는 개념으로 이해할

수 있다. 수학적으로는 임의의 양수 a 와 0 이상의 정수 n 에 대해 다음과 같이 표현된다.

$${}_na = \begin{cases} 1 & (n=0) \\ a^{({}_{n-1}a)} & (n \geq 1) \end{cases}$$

테트레이션은 거듭제곱을 여러 차례 수행하는 연산이므로, 적은 연산 횟수만으로도 상상할 수 없을 정도로 큰 값을 얻을 수 있다. 이처럼 어떠한 수학적 모델이 시간이 지남에 따라 급격히 성장하는 양상을 보일 때 ‘지수적으로 증가한다’라는 표현을 사용한다.

나) 테트레이션 그래프

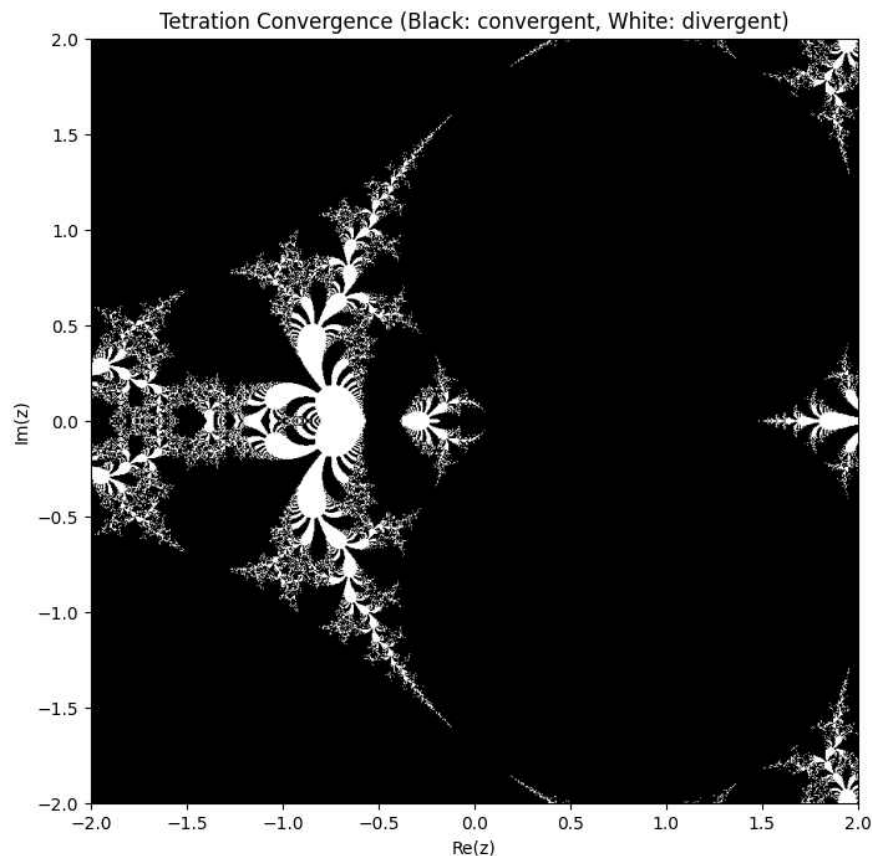


그림 2 테트레이션 그래프

이 그래프는 복소수 입력에 대한 테트레이션 함수의 값을 시각적으로 표현한 것이다. 복소 평면상에서 테트레이션 함수가 수렴하는 지점을 검정색으로 나타내고, 발산하는 지점을 하얀색으로 표시하면 [그림 2] 같은 형태가 나타난다. 특정 지점에서는 수렴 및 발산의 명확한 경계를 구분할 수 없으며 무한한 복잡성이 나타나기도 한다.

3) 선행연구

가) ECB(electronic codebook)

가장 간단한 블록 암호 운용 방식으로, 메시지를 여러 블록으로 나누어 각 블록을 독립적으로 암호화하는 구조를 가진다. 모든 블록이 동일한 암호화 키를 사용하기 때문에 보안에 취약하며, 특히 동일한 블록이 암호화될 경우 같은 암호문이 생성되어 반복 공격에 쉽게 노

출된다. 또한, 블록 단위로 순차적으로 암호화되므로 하나의 블록이 해독되면 나머지 블록도 해독될 위험이 있다.

암호문이 블록 크기의 배수가 되어야 하므로, 복호화 후 평문을 정확히 얻기 위해 패딩이 필요하다. 이때 패딩은 부족한 데이터를 특정 크기로 맞추기 위해 추가하는 작업을 의미한다. ECB의 장점은 각 블록이 독립적으로 동작하므로 한 블록에서 오류가 발생해도 다른 블록에는 영향을 미치지 않는다는 점이다. 그러나 해당 블록의 오류가 그대로 유지된다는 한계가 있다.

나) CBC(cipher-block chaining)

CBC는 블록 암호화의 대표적인 운용 방식 중 하나이다. 각 블록이 암호화되기 전에 이전 블록의 암호화 결과와 XOR 연산을 수행한 후 암호화하는 방식이다. 첫 번째 블록은 이전 블록이 없기 때문에, 대신 초기화 벡터가 사용된다. 동일한 키와 평문으로 여러 번 암호화하더라도 각기 다른 암호문이 생성되도록 초기화 벡터는 매 암호화마다 다른 값을 사용해야 한다. 복호화하기 위해선 암호화에 사용된 초기화 벡터를 알아야 하며, 알 수 없는 경우 고정된 초기화 벡터를 사용한다.

이는 보안성이 높은 암호화 기법으로 널리 사용되지만, 암호화 과정이 이전 블록의 결과에 의존하므로 병렬 처리가 불가능하며, 순차적으로 처리해야 한다. 그러나 복호화는 각 블록을 복호화한 후 XOR 연산을 통해 평문을 복구할 수 있기에 병렬 처리가 가능하다. 또한, 암호문이 블록의 배수가 되어야 하므로, 패딩 작업이 필요하다. CBC 방식은 암호문이 손상될 경우 해당 블록뿐만 아니라 다음 블록의 평문에도 영향을 미친다는 단점이 존재한다. 이를 고려하여 실생활에서는 적절한 초기화 벡터 관리와 패딩 방식을 적용하여 보안성을 강화하는 것이 중요하다.

3. 연구 방법

가. 주요 알고리즘 구현

테트레이션은 반복적인 거듭제곱 연산으로 정의된다. 복소수 영역에서 이 연산을 반복 적용했을 때, 결과가 수렴 여부를 확인하여 해당 좌표의 값이 수렴하면 검정, 발산하면 하얀색으로 그래프를 나타낸다.

```
# 테트레이션 반복함수 정의
def tetration(z, max_iter=50, threshold=1e6):
    val = z
    for _ in range(max_iter):
        try:
            val = z ** val
        except OverflowError:
            return np.nan
    if abs(val) > threshold:
```

```

        return np.nan
    return val

```

위의 코드와 같이 지정된 최대 반복 횟수보다 반복했음에도 결과가 일정하면 수렴으로, 특정 값을 넘는 절댓값이 발생하면 발산으로 간주한다.

```

# 수렴과 발산 여부를 확인해 결과를 배열에 저장
results = np.zeros(Z.shape, dtype=np.complex128)

for i in range(Z.shape[0]):
    for j in range(Z.shape[1]):
        results[i, j] = tetration(Z[i, j])

# 결과에 따라 색깔 결정 (수렴하면 검정, 발산하면 하양)
colors = np.full(Z.shape + (3,), 1.0) # 기본값: 하양(발산)
colors[np.isfinite(results)] = [0,0,0] # 수렴하면 검정

```

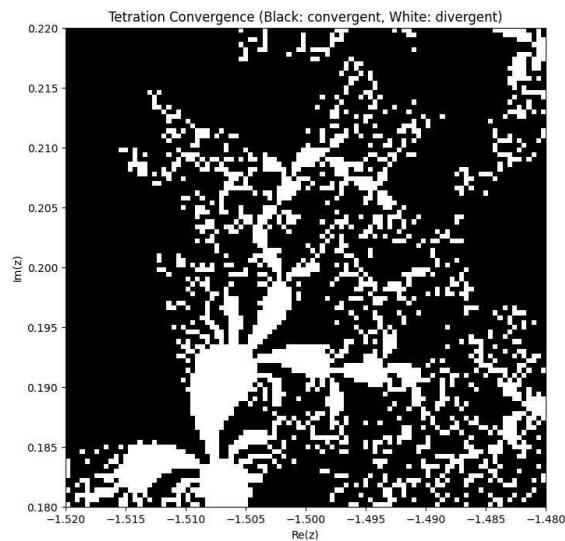


그림 3 테트레이션 그래프

생성된 테트레이션 그래프의 이미지를 불러온 이미지의 픽셀에 모두 일대일 대응되도록 해상도를 조절한다.

```

# 고해상도 테트레이션 마스크 생성 함수
def generate_tetration_mask(width, height, center_x, center_y, zoom,
resolution_multiplier=2):
    """Generates the high-resolution tetration convergence/divergence pattern."""
    high_width, high_height = width * resolution_multiplier, height *
resolution_multiplier
    range_x, range_y = 2 / zoom, 2 / zoom
    re = np.linspace(center_x - range_x, center_x + range_x, high_width)
    im = np.linspace(center_y - range_y, center_y + range_y, high_height)

```

```

Re, Im = np.meshgrid(re, im)
Z = Re + Im * 1j

    print(f"테트레이션 계산 중 (해상도: {high_width}x{high_height}, 원본의
{resolution_multiplier}배)...")
    v_tetration = np.vectorize(tetration)
    results = v_tetration(Z)
    high_res_mask = np.isfinite(results).astype(int)
    print("계산 완료.")

    print("고해상도 마스크를 원본 크기로 다운샘플링 중...")
    mask = np.zeros((height, width), dtype=int)
    for i in range(height):
        for j in range(width):
            region =
high_res_mask[i*resolution_multiplier:(i+1)*resolution_multiplier,
j*resolution_multiplier:(j+1)*resolution_multiplier]
            values, counts = np.unique(region, return_counts=True)
            mask[i, j] = values[np.argmax(counts)]
    return mask

```

나. 암호화 단계

- 1) 암호화할 이미지를 선택한다
- 2) 테트레이션 그래프에서 좌표와 확대비율을 선택하고 그래프의 해상도를 이미지와 동일하게 조정하여 각각의 픽셀이 모두 일대일 대응되도록 한다.
- 3) 테트레이션 그래프에서 검정색인 픽셀에 대응되는 이미지의 픽셀은 원래 색의 반대색으로 변환한다.
- 4) 반대로 흰색 픽셀에 대응되는 이미지의 픽셀은 원래 색을 그대로 유지한다.
- 5) 서로 다른 테트레이션 그래프들을 이용하여 여러 번 반복하여 암호화한다.

다. 복호화

테트레이션 자체가 매우 큰 연산이고 이를 복소수 범위에서 여러 차례 연산한다. 이에 연산 결과에서 원래 복소수를 역연산하여 구하는 것은 불가능에 가깝고, 이는 암호화에 매우 적합하다. 암호화된 사진을 암호화와 같은 방법을 한 번 더 적용하면 변환되었던 색은 다시 변환되어 원래 색으로 돌아오고, 변환되지 않은 색은 여전히 유지되면서 처음 이미지를 다시 얻을 수 있다. 즉, 키로 설정된 복소평면의 좌표와 확대비율을 통해 복호화가 가능하다.

4. 연구 결과

본 연구에서는 테트레이션을 이용한 새로운 암호화 방식의 개념을 정립하고, 이를 기반으로 한 키 생성 방법과 암호화/복호화 과정을 설계하였다. 설계된 암호화 과정을 적용시켜 보자.

암호학의 역사를 바꾸는 테트레이션 연산 기반 암호화 연구

그림 4 암호화 시킬 이미지

위의 이미지를 설계한 방법을 통해 암호화하면 다음과 같다.

Layer 1: Center(-1.9076, 0.1317) Zoom: 3.6845
Layer 2: Center(-1.1795, -0.4275) Zoom: 2.8890
Layer 3: Center(-1.4103, -0.1905) Zoom: 4.5755
Layer 4: Center(-1.5612, 0.0960) Zoom: 3.9895
Layer 5: Center(-1.5904, -0.0267) Zoom: 4.8734
Layer 6: Center(-1.6219, 0.2245) Zoom: 4.3829
Layer 7: Center(-1.3159, 0.1915) Zoom: 3.2491
Layer 8: Center(-1.5280, -0.1449) Zoom: 3.4348
Layer 9: Center(-1.4897, 0.2192) Zoom: 1.9140
Layer 10: Center(-1.2812, -0.0573) Zoom: 2.7663



그림 5 암호화 후 이미지(반복 횟수: 10)

위의 이미지를 암호화 역순으로 다시 알고리즘을 적용하여 복호화가 가능하다.

테트레이션 암호는 지수적 증가보다 훨씬 빠르게 증가하는 연산을 기반으로 하여 키 공간을 극도로 확대할 수 있다. 또한 테트레이션은 수학적으로 역함수가 정의되지 않아 키 없이 복호화가 불가능하므로 보안성이 우수하다. 그러나 테트레이션 수렴성 계산은 고정밀 연산이 필요하기 때문에 다른 이미지 암호화에 비해 느리다는 단점이 있다. 또한 동일 이미지에 대해 다른 키를 사용할 경우 암호화 결과가 완전히 달라지므로 키의 독립성이 강하다.

5. 기대되는 성과

본 연구를 통해 테트레이션 기반 암호화 기법이 기존 암호화 방식과 차별화된 보안성을 제공할 수 있으며, 다양한 응용 분야에서 활용될 수 있다. 테트레이션의 수렴과 발산 패턴을 이용한 테트레이션 기반 암호화 키 생성 방식은 기존 선형적 또는 다항식 기반 암호화 기법과는 다른 접근법을 제시한다. 특히, 테트레이션 함수가 가지는 비선형성과 복잡성으로 인해 키의 패턴을 예측하기 어렵다.

이런 방식은 메시지 암호화 및 보안 통신, 디지털 서명, 인증 시스템 등 다양한 보안 분야에 적용될 수 있으며, 특히 프랙탈 기반 시각적 암호화 기법과 결합하여 정보 보안 강화에 활용될 수 있다. 텍스트뿐만 아니라 이미지 및 영상 데이터를 암호화하는 데 응용할 수 있어, 군사, 정부 기관, 금융 보안 등 높은 보안이 요구되는 환경에서도 유용하게 사용될 수 있다. 그러나 테트레이션 함수의 계산이 비교적 복잡하고 암호화 및 복호화 과정에서 연산 효율성 문제가 제기될 수 있다. 이런 약점을 보완한다면 테트레이션 기반 암호화 방식이 새로운 보안 기술로 자리 잡을 수 있을 것이다. 향후 연구를 통해 테트레이션 기반 암호화 방식의 보안성을 정량적으로 분석하고, 기존 암호화 기법과 테트레이션 기반 암호화 기법의 성능을 비교할 것이다. 특히, 테트레이션의 수렴/발산 패턴의 무작위성을 충분히 갖추고 있는지 검증하고자 한다. 또한, 테트레이션을 활용한 시각적 암호화 기법의 실용성을 평가하고, 이를 실제 보안 시스템에 적용할 수 있는 가능성을 분석할 것이다.

6. 참고문헌

- [1] 양자내성암호 소프트웨어 구현 동향 (권혁동, 2024)
- [2] 이미지 데이터 암호화를 위한 인터넷 통신 오버헤드 분석 (송원석, 김재혁, 김종찬, 2024)
- [3] 어텐션 시각화를 통한 암호화 트래픽 분석 (김주성, 장윤성, 박지태, 최유진, 백의준, 2024)